

ESPECIFICACIÓN DE PROTOCOLOS Y SECUENCIA DE LLAMADAS

Diagrama de Flujo y Secuencia de Llamadas de la Plataforma

Mapeo detallado de interacciones cliente-servidor, llamadas asíncronas, eventos Realtime, contratos de payloads, manejo de concurrencia y cadena de llamadas de extremo a extremo.

VERSIÓN DEL SISTEMA

2.0.0

FECHA DE EMISIÓN

Septiembre 2026

AUTOR / RESPONSABLE

Equipo de Integración, APIs y Backend Engineering

CLASIFICACIÓN

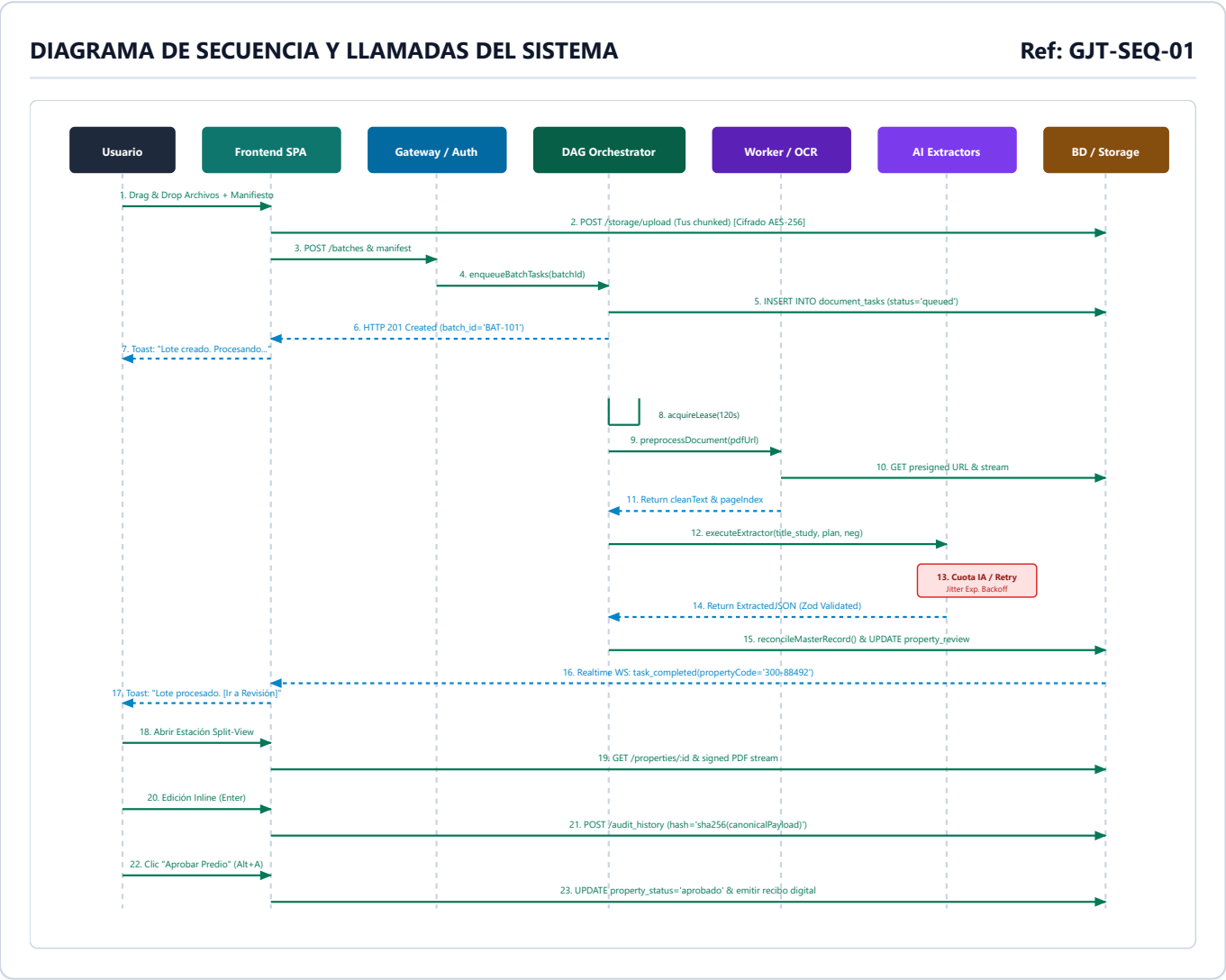
Técnica Oficial / Confidencial

1. Ciclo de Vida de Peticiones y Secuencia Técnica

La arquitectura de **Grupo Jurídico Territorium** se fundamenta en un modelo de **comunicación asíncrona reactiva con garantías de no bloqueo**. A diferencia de soluciones monolíticas tradicionales donde una petición HTTP se mantiene abierta durante 60 segundos mientras un LLM procesa un documento (provocando timeouts y pérdida de control), Territorium separa de forma tajante la **confirmación de ingesta** de la **ejecución distribuida del trabajo**.

2. Diagrama de Secuencia End-to-End

El siguiente diagrama detalla las 23 etapas secuenciales desde que el usuario arrastra un lote de expedientes hasta que se aprueba el predio y se emite el registro forense inmutable:



3. Especificación Detallada de Llamadas y Endpoints REST

PASO	ENDPOINT / MÉTODO	COMPONENTE ORIGEN → DESTINO	ESTRUCTURA DEL PAYLOAD	GARANTÍA & RESILIENCIA
2	POST /storage/v1/upload/resumable (Protocolo Tus)	Frontend → S3 Storage	Chunks binarios de 5MB con cabecera Upload- Length y hash SHA-256.	Reanudación automática ante micro-cortes de red sin reiniciar subida.
3	POST /rest/v1/batches	Frontend → Gateway API	{ projectId, batchName, totalFiles, manifestJson }	Validación de schema Zod y asignación de ID determinístico BAT- XXX.
8	POST /rpc/acquire_pending_tasks	DAG Engine → PostgreSQL	{ workerId, maxBatchSize: 10, leaseSeconds: 120 }	Uso de SELECT ... FOR UPDATE SKIP LOCKED para evitar bloqueos.
12	POST /v1/chat/completions (AI Engine)	DAG Engine → Modelo LLM	Prompt estructurado + Schema JSON estricto de extracción jurídica.	Circuit breaker con 3 reintentos exponenciales y fallback de modelo.
16	WebSocket /realtime/v1/websocket	PostgreSQL → Frontend	Evento task_completed con folio, tiempo de proceso y estado.	Reconexión automática con exponential backoff y heartbeat de liveness.
21	POST /rest/v1/audit_attribute_history	Split-View → Base de Datos	{ folio, attribute, previousVal, newVal, sha256Hash, author }	Inserción en tabla inmutable de auditoría forense append-only.

4. Contratos de Comunicación y Esquemas de Carga Útil (Payloads)

4.1. Payload de Extracción del Agente Estudio de Títulos

```
{
  "propertyCode": "PRD-2026-0045",
  "titleStudy": {
    "folioMatricula": "300-88492",
    "registeredAreaM2": 12450.00,
    "currentOwner": { "name": "Hacienda El Vergel S.A.S.", "nit": "900123456-7" },
    "liens": [{ "type": "HIPOTECA", "entity": "Banco Agrario", "active": true }],
    "confidenceScore": 0.94
  }
}
```

4.2. Payload de Conciliación Tripartita del Motor Matemático

```
{
  "folioMatricula": "300-88492",
  "areaComparison": {
    "titleAreaM2": 12450.00,
    "planAreaM2": 11800.00,
    "discrepancyPercentage": 5.22,
    "isWithinTolerance": true, // Umbral permitido: 15.0%
    "severity": "admissible_tolerance"
  },
  "perimetralDiscrepancies": []
}
```

5. Política de Manejo de Concurrencia y Retenciones (Leasing)

Para evitar colisiones entre múltiples instancias de workers que intenten procesar la misma tarea simultáneamente en entornos de escalado horizontal, Territorium implementa un **patrón de arrendamiento distribuido (Distributed Lease Pattern)**:

Adquisición Atómica: Cada worker ejecuta la función `acquirePendingTasks` que marca la tarea con `leaseExpiresAt = NOW() + 120s` y asigna su `workerId`.

Heartbeat de Renovación: Si un documento escaneado de 100 páginas requiere más de 120 segundos de OCR intensivo, el worker renueva su lease cada 45 segundos mientras la tarea permanezca en ejecución activa.

Recuperación de Tareas Huérfanas (Zombie Task Reaper): Si un worker colapsa físicamente o se reinicia el pod de Kubernetes, el orquestador detecta tareas con `leaseExpiresAt < NOW()` y las libera automáticamente a la cola para que otro worker las retome de forma transparente.